

0. Metryczka

- **Uczelnia:** Politechnika Białostocka
- **Wydział:** Wydział Informatyki
- **Kierunek:** Informatyka
- **Przedmiot:** Inżynieria Oprogramowania
- **Semestr studiów:** Semestr IV
- **Prowadzący:** dr M. Czajkowski
- **Data przekazania sprawozdania:** 14.06.2026

0.1. Skład zespołu i podział pracy

- Przemysław Pieszko: Przypadek użycia "Połączenie konta Steam/Discord", diagramy czynności i przebiegu dla tego procesu, diagram stanu dla integracji zewnętrznych.
- Karol Ziemak: Przypadek użycia "Dopasowanie użytkowników (Match)", diagramy czynności i przebiegu dla matchmakingu, diagram stanu dla procesu dopasowania.
- Kamil Osakowicz: Przypadek użycia "Zgłoszenie użytkownika", moduł moderacji, diagramy czynności, przebiegu oraz maszyna stanów dla cyklu życia zgłoszenia.

0.2. Proponowana punktacja

- Przemysław Pieszko: 100%
- Karol Ziemak: 100%
- Kamil Osakowicz: 100%

0.3. Adres (publiczny) repozytorium projektu

- **URL:** <https://github.com/karlosiwo/ProjektIO>
- **Dane dostępne:** Repozytorium jest publiczne.

1. Treść zadania projektowego

- Aplikacja GamerMatch jest systemem klasy platformy matchmakingowej.
- Jej głównym zadaniem jest łączenie graczy.
- Proces łączenia odbywa się na podstawie ich preferencji, aktywności oraz danych zewnętrznych.

- Obsługiwane zewnętrzne platformy danych to m.in. Steam oraz Discord.

2. Cel budowania systemu, jego zakres oraz kontekst

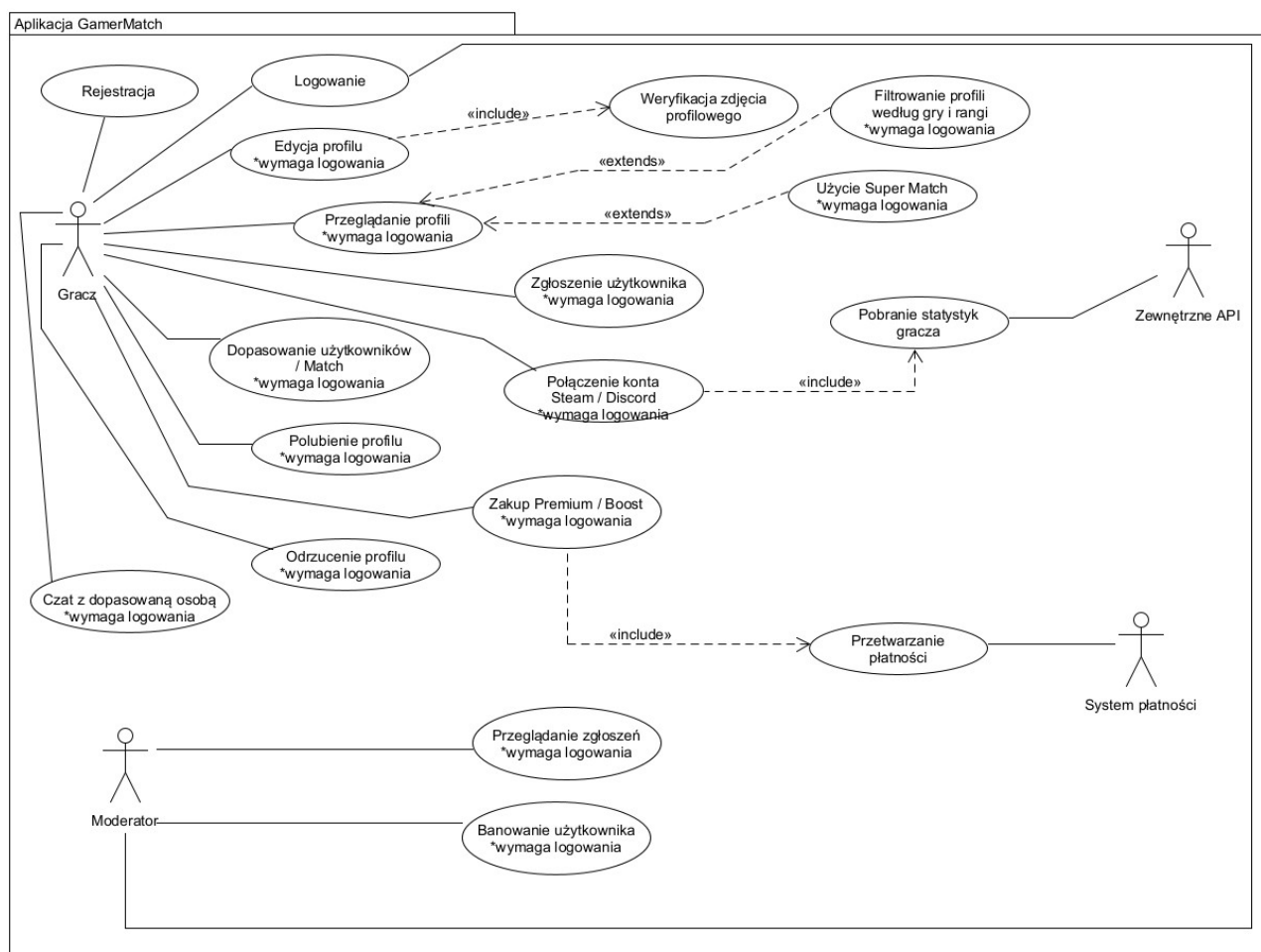
- **Cel i kontekst:** Zbudowanie centralnej platformy społecznościowej dla graczy, integrującej profile z wielu ekosystemów gamingowych w celu automatyzacji poszukiwania partnerów do gry o zbliżonym poziomie i preferencjach.
- **Mierzalne korzyści z wdrożenia:**
 - Skrócenie średniego czasu znalezienia partnera do wspólnej gry o minimum 60% (estymowane przejście z manualnego poszukiwania na forach/Discordach trwającego ok. 15 minut do "Matchu" w aplikacji w czasie 2-5 minut).
 - Osiągnięcie miesięcznego przychodu rzędu 2 000 000 PLN w pierwszym roku po wdrożeniu funkcji premium.
- **Niemierzalne korzyści z wdrożenia:**
 - Lepsza jakość dopasowań w systemie.
 - Możliwość natychmiastowego rozpoczęcia rozmowy po dopasowaniu z graczem o podobnych zainteresowaniach.
 - Zwiększenie bezpieczeństwa i jakości społeczności.
 - Eliminacja toksycznych zachowań.

3. Słownik

- **Match:** Wzajemne polubienie profili przez dwóch użytkowników, skutkujące utworzeniem dopasowania i odblokowaniem czatu.
- **Super Match:** Funkcja premium zwiększająca priorytet dopasowania dla wybranego profilu oraz szansę na szybsze połączenie.
- **Boost:** Opcja premium oferująca zwiększoną widoczność profilu gracza w module przeglądania.
- **Zewnętrzne API:** Interfejs programistyczny platform (np. Steam, Discord) umożliwiający weryfikację rang i osiągnięć.

4. Perspektywa przypadków użycia

4.1. Diagramy przypadków użycia



4.1.1. Opisy tekstowe wszystkich aktorów

- **Gracz:** Główny aktor systemu inicjujący podstawowe akcje takie jak: rejestracja, przeglądanie profili, łączenie kont zewnętrznych, polubienia oraz komunikacja na czacie.
- **Moderator:** Aktor zarządzający odpowiedzialny za bezpieczeństwo platformy. Przegląda zgłoszenia trafiające do systemu i podejmuje działania administracyjne, w tym banowanie użytkowników.
- **Zewnętrzne API:** Aktor systemowy dostarczający na żądanie dane profilowe, statystyki i rangi graczy z platform powiązanych.
- **System płatności:** Zewnętrzny interfejs obsługujący transakcje związane z zakupem funkcji Premium i Boostów.

4.1.2. Opisy tekstowe wybranych przypadków użycia

Przypadek 1: Połączenie konta Steam/Discord

- **Nazwa przypadku:** Połączenie konta Steam/Discord
- **Aktorzy:** Gracz, Zewnętrzne API

- **Podstawowy ciąg zdarzeń:**

- System wyświetla opcję integracji kont w profilu użytkownika.
- Gracz wybiera platformę (np. Steam lub Discord) do połączenia.
- System przekierowuje użytkownika do zewnętrznego serwisu w celu autoryzacji.
- Gracz loguje się i wyraża zgodę na dostęp do danych.
- System nawiązuje połączenie z wykorzystaniem Zewnętrznego API.
- System pobiera dane gracza (np. statystyki, ranga, osiągnięcia).
- System aktualizuje profil użytkownika i wyświetla status "Connected".

- **Alternatywne ciągi zdarzeń:**

- Błąd połączenia z API: System nie może pobrać danych; informuje Gracza o błędzie i umożliwia ponowienie próby.
- Odmowa autoryzacji: Gracz nie wyraża zgody na dostęp do danych; system przerywa proces i powraca do edycji profilu.
- Niepoprawne dane logowania: System informuje o błędzie logowania w serwisie zewnętrznym; użytkownik może ponowić próbę.

- **Zależności czasowe:**

- Częstotliwość wykonania: sporadyczna (zwykle jednorazowa podczas konfiguracji profilu).
- Przewidywane spiętrzenia: brak istotnych spiętrzeń.
- Typowy czas realizacji: 30-60 sekund.
- Maksymalny czas realizacji: zależny od czasu odpowiedzi zewnętrznego API.

- **Wartości uzyskiwane po zakończeniu:**

- Zweryfikowane dane gracza (ranga, aktywność, osiągnięcia).

- Lepsza jakość dopasowań w systemie.

Przypadek 2: Dopasowanie użytkowników (Match)

- **Nazwa przypadku:** Dopasowanie użytkowników (Match)
- **Aktorzy:** Gracz
- **Podstawowy ciąg zdarzeń:**
 - Gracz uruchamia moduł przeglądania profili.
 - System wyświetla profile innych graczy na podstawie preferencji i statystyk.
 - Gracz analizuje profil i wybiera akcję „Polubienie”.
 - System zapisuje decyzję użytkownika.
 - System sprawdza, czy drugi użytkownik również polubił profil Gracza.
 - W przypadku wzajemnego polubienia system tworzy dopasowanie (match).
 - System wyświetla komunikat „It's a Match!” oraz ekran dopasowania.
 - System umożliwia rozpoczęcie czatu z dopasowanym użytkownikiem.
- **Alternatywne ciągi zdarzeń:**
 - Odrzucenie profilu: Gracz wybiera opcję „Odrzucenie”. System usuwa profil z widoku i wyświetla kolejny.
 - Brak wzajemnego dopasowania: Drugi użytkownik nie polubił profilu Gracza. System zapisuje decyzję i kontynuuje proces przeglądania.
 - Użycie funkcji Super Match: Gracz wybiera opcję „Super Match”. System zwiększa priorytet dopasowania dla wybranego profilu.
- **Zależności czasowe:**
 - Częstotliwość wykonania: bardzo wysoka (wielokrotnie w trakcie jednej sesji).
 - Przewidywane spiętrzenia: godziny wieczorne, weekendy.
 - Typowy czas realizacji: kilka-kilkanaście sekund na jeden profil.

- Maksymalny czas realizacji: nieokreślony.
- **Wartości uzyskiwane po zakończeniu:**
 - Dopasowanie z graczem o podobnych zainteresowaniach i poziomie.
 - Możliwość rozpoczęcia rozmowy.

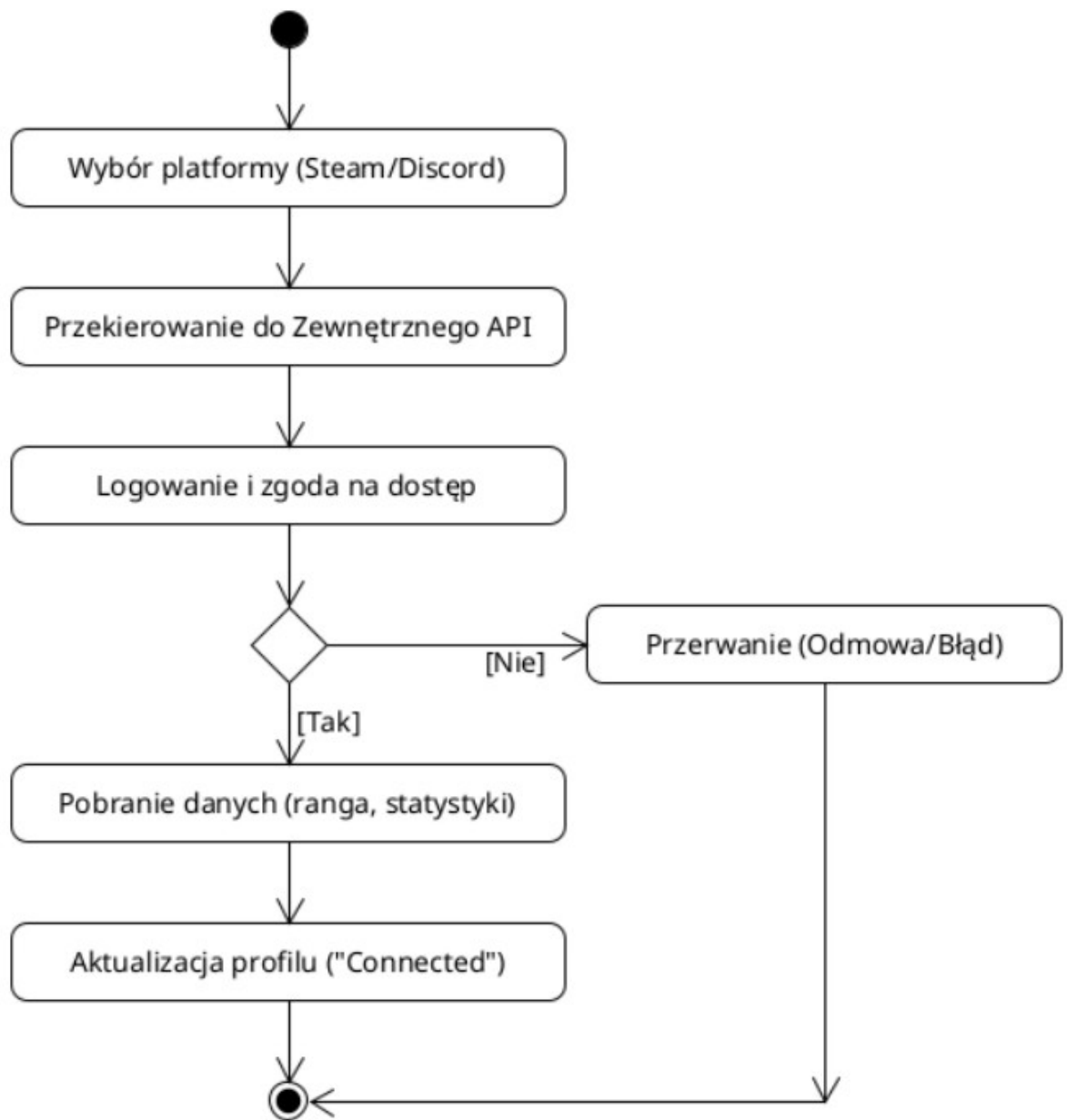
Przypadek 3: Zgłoszenie użytkownika

- **Nazwa przypadku:** Zgłoszenie użytkownika
- **Aktorzy:** Gracz, Moderator
- **Podstawowy ciąg zdarzeń:**
 - Gracz wybiera opcję „Zgłoszenie użytkownika” (np. w profilu lub czacie).
 - System wyświetla formularz zgłoszenia z listą powodów.
 - Gracz wybiera powód i zatwierdza zgłoszenie.
 - System zapisuje zgłoszenie w bazie danych.
 - System przeprowadza automatyczną analizę treści (AI).
 - Zgłoszenie trafia do kolejki w module dla Moderadora.
 - Moderator analizuje zgłoszenie i dostępne dowody.
 - Moderator podejmuje decyzję (np. ban lub ograniczenie konta).
 - System zapisuje decyzję i aktualizuje status użytkownika.
- **Alternatywne ciągi zdarzeń:**
 - Odrzucenie zgłoszenia: Moderator uznaje zgłoszenie za bezpodstawne. System zamyka zgłoszenie bez konsekwencji dla użytkownika.
 - Automatyczne ograniczenie konta: System wykrywa dużą liczbę zgłoszeń. System tymczasowo ogranicza widoczność profilu.
 - Błąd systemu: System nie może zapisać zgłoszenia i informuje użytkownika o problemie.
- **Zależności czasowe:**
 - Częstotliwość wykonania: zależna od aktywności użytkowników.

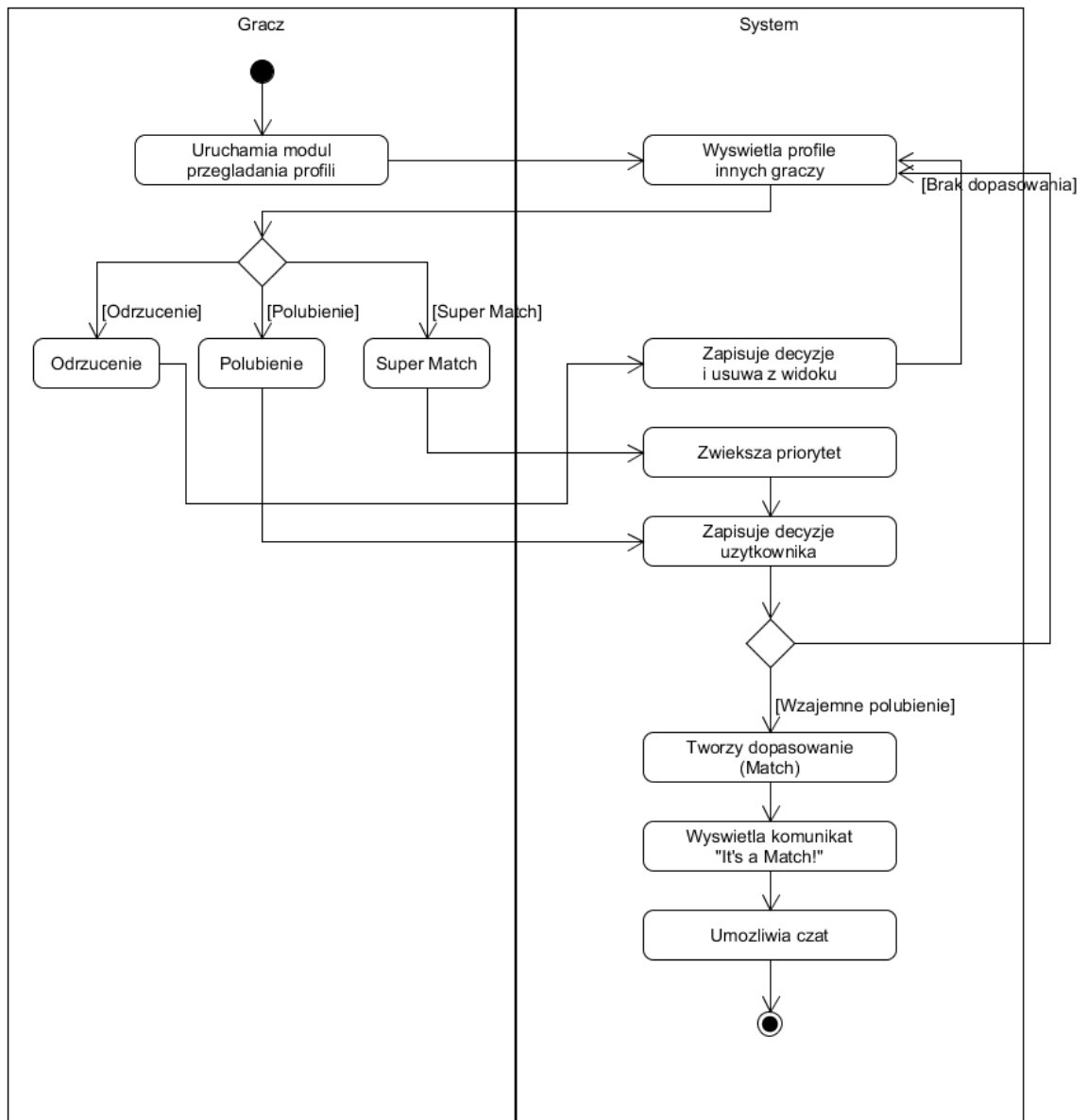
- Przewidywane spiętrzenia: godziny wieczorne, duża aktywność użytkowników.
- Typowy czas realizacji: zgłoszenie ~30 sekund, analiza kilka-kilkanaście minut.
- Maksymalny czas realizacji: do kilku godzin (w zależności od kolejki zgłoszeń).
- **Wartości uzyskiwane po zakończeniu:**
 - Zwiększenie bezpieczeństwa i jakości społeczności.
 - Eliminacja toksycznych zachowań.

4.2. Diagramy czynności

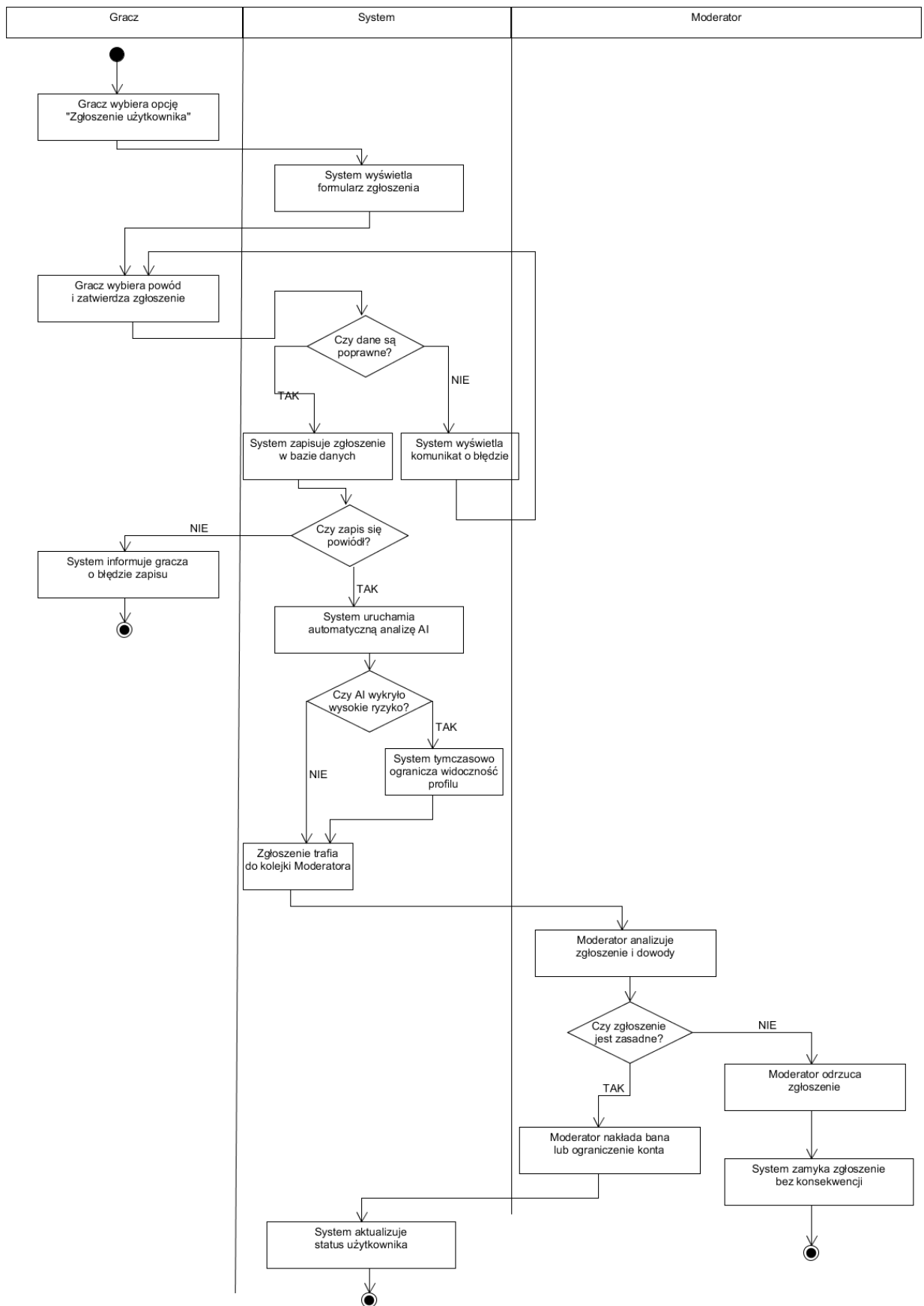
- **Diagram 1: Połączenie konta Steam/Discord**



- **Diagram 2: Dopasowanie użytkowników (Match)**

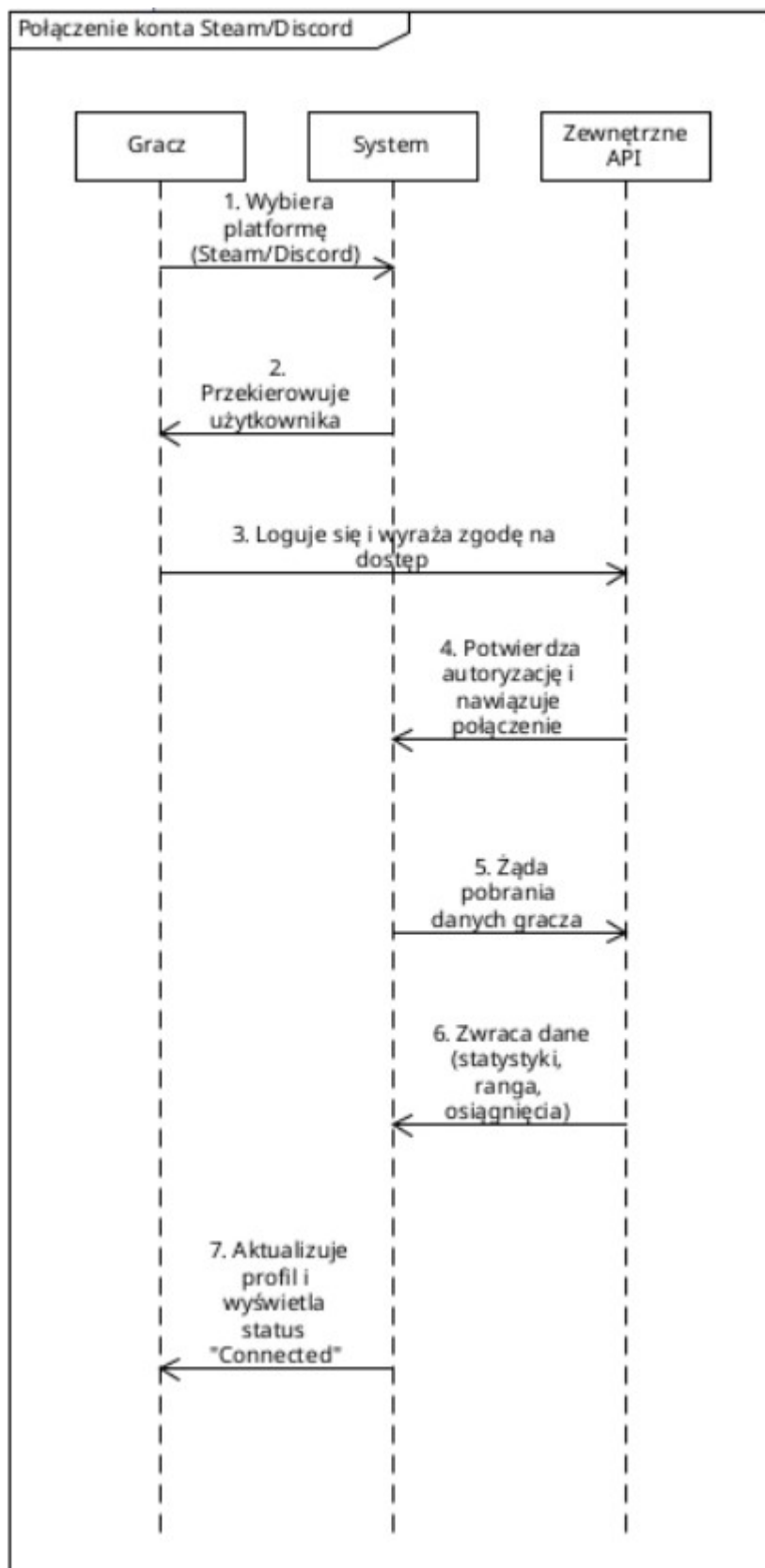


• Diagram 3: Zgłoszenie użytkownika

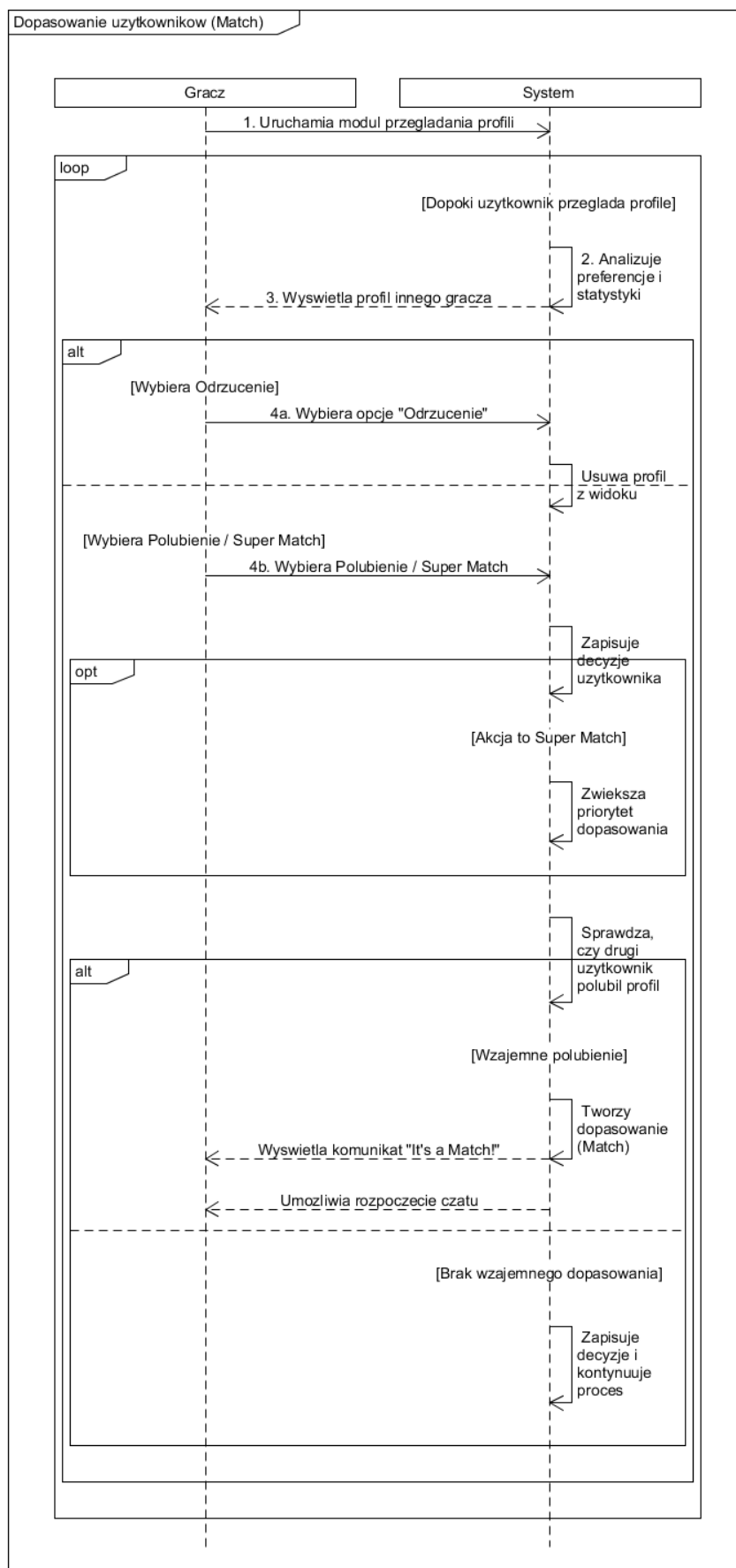


4.3. Diagramy interakcji (przebiegu)

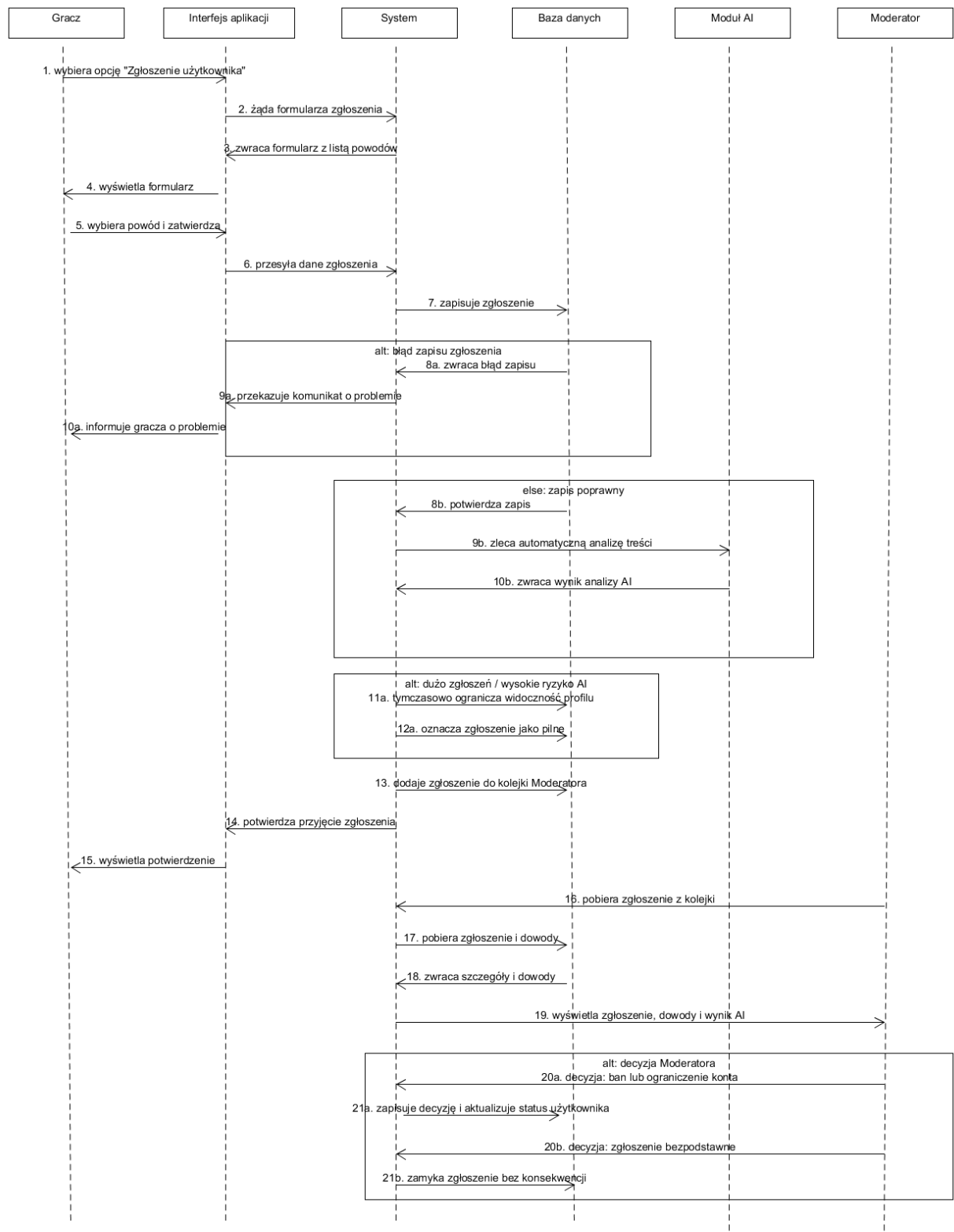
- Diagram 1: Połączenie konta Steam/Discord



- **Diagram 2: Dopasowanie użytkowników (Match)**



• **Diagram 3: Zgłoszenie użytkownika**



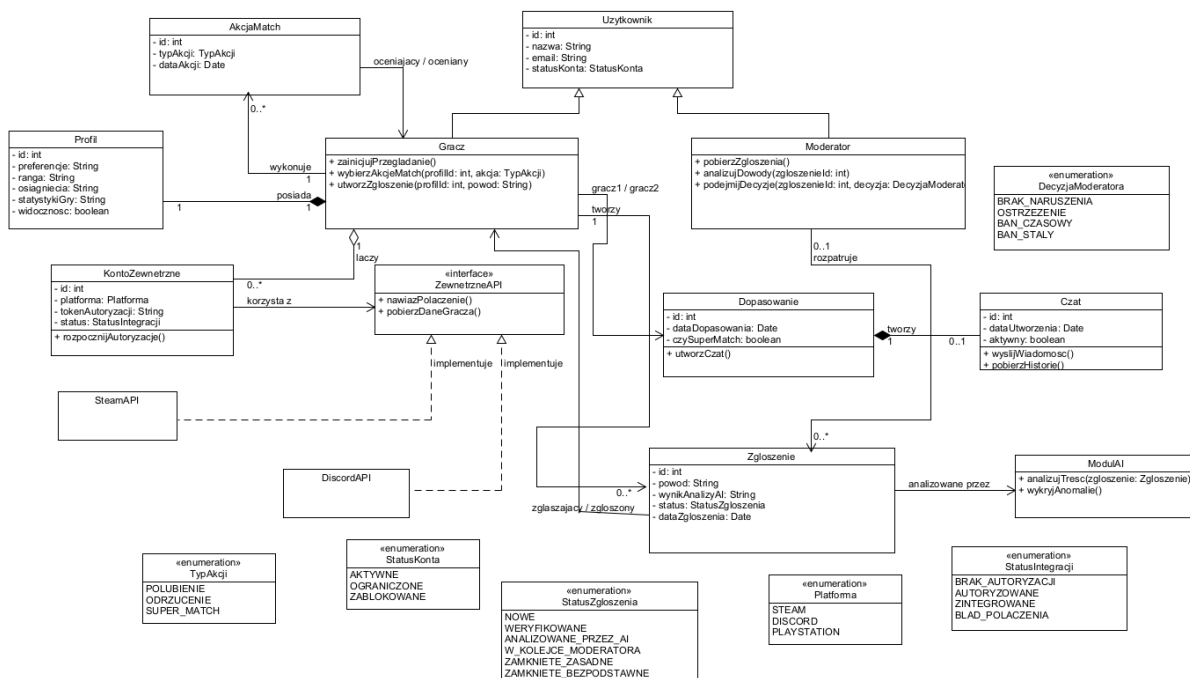
5. Perspektywa projektowa

5.0. Proponowana architektura systemu

Zaproponowano architekturę mikrousługową (Microservices Architecture) w chmurze (Cloud). Logika backendowa obsługiwana jest przez usługi napisane w języku Java (Spring Boot) w celu

łatwego skalowania komponentów takich jak silnik Matchmakingu, Czat oraz integracja API. Frontend stanowić będzie wieloplatformowa aplikacja mobilna (iOS/Android). Usługi połączone są z klastrem relacyjnych (PostgreSQL) baz danych dla profili i transakcji.

5.1. Diagram klas



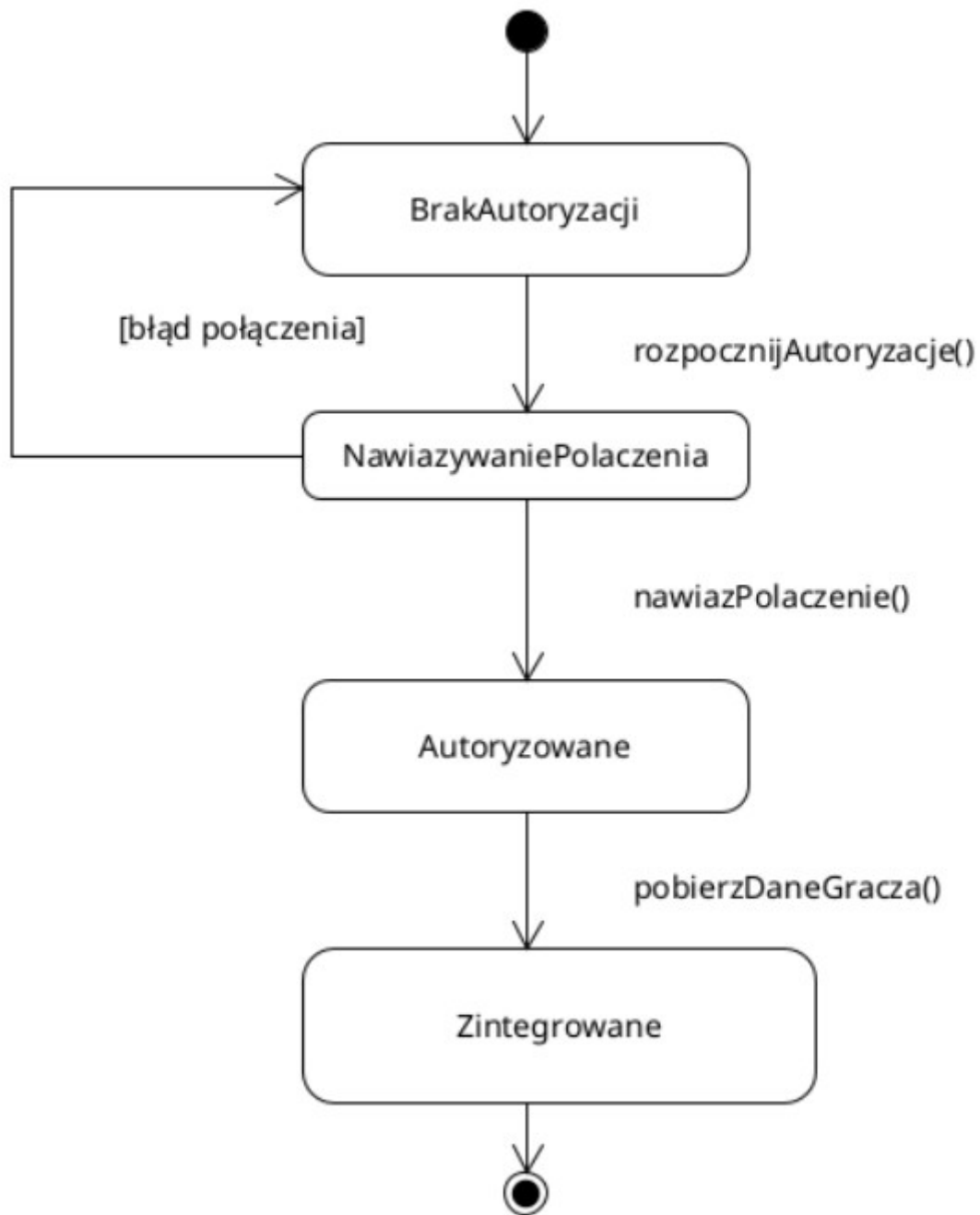
5.2. Uporządkowany alfabetycznie wykaz wszystkich klas

- **AkcjaMatch (Asynchroniczna decyzja użytkownika)**
 - Atrybuty: oceniajacyId: int, ocenianyId: int, typAkcji: TypAkcji (polubienie/odrzucenie)
- **Dopasowanie (Instancja udanego Matchu)**
 - Atrybuty: id: int, gracz1Id: int, gracz2Id: int, dataDopasowania: Date, czySuperMatch: boolean
 - Metody: utworzCzat()
- **Gracz (Główny aktor)**
 - Atrybuty: id: int, nazwa: String, statusKonta: String
 - Metody: zainicjujPrzeglądanie(), wybierzAkcjeMatch(profilId: int, akcja: String), utworzZgloszenie(profilId: int, powod: String)
- **Interfejs ZewnetrzneAPI (Most komunikacyjny)**

- Metody: `nawiazPolaczenie()`, `pobierzDaneGracza()`
- **KontoZewnetrzne (Integracja platform)**
 - Atrybuty: `platforma: String`, `tokenAutoryzacji: String`
 - Metody: `rozpocznijAutoryzacje()`
- **Moderator (Aktor zarządzający)**
 - Atrybuty: `id: int`, `nazwa: String`
 - Metody: `pobierzZgloszenia()`, `analizujDowody(zgloszenieId: int)`, `podejmijDecyzje(zgloszenieId: int, decyzja: String)`
- **ModulAI (Zautomatyzowany silnik analizy)**
 - Metody: `analizujTresc(zgloszenie)`, `wykryjAnomalie()`
- **Profil (Reprezentacja wizualna i statystyczna)**
 - Atrybuty: `id: int`, `graczId: int`, `preferencje: String`, `ranga: String`, `osiagniecia: String`, `statystykiGry: String`, `statusPolaczenia: String`, `widocznosc: boolean`
- **Zgloszenie (Encja systemu moderacji)**
 - Atrybuty: `id: int`, `zgłaszającyId: int`, `zgłoszonyId: int`, `powod: String`, `wynikAnalizyAI: String`, `status: String`

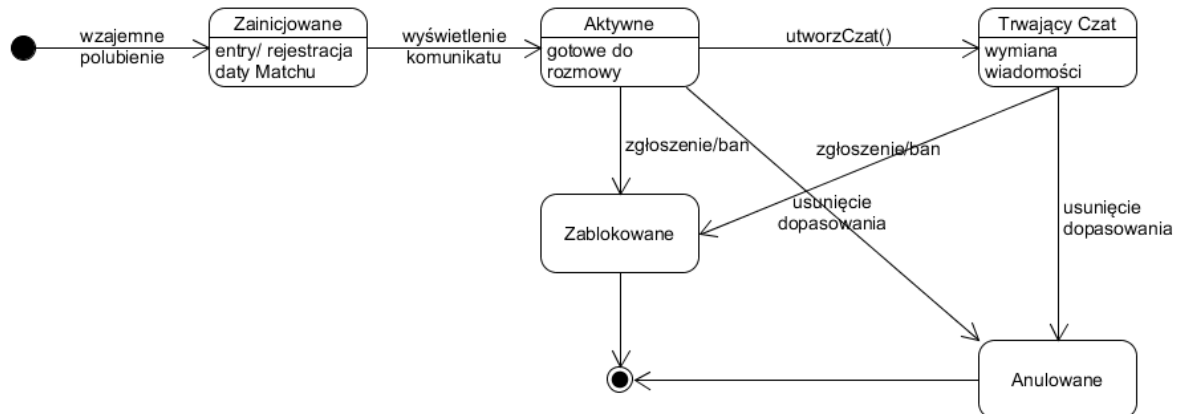
5.3. Diagramy stanów

- **Diagram 1: Cykl życia dla instancji klasy KontoZewnetrzne (Połączenie konta Steam/Discord)**



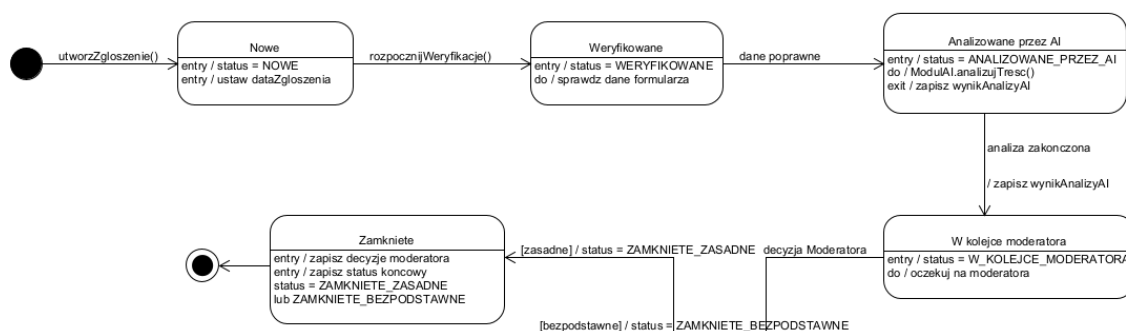
- **BrakAutoryzacji**: Stan bazowy integracji.
- **NawiazywaniePolaczenia**: Stan przejściowy. Oczekuje na token zwrotny (w przypadku błędu połączenia system wraca do stanu bazowego).
- **Autoryzowane**: Zewnętrzne API pomyślnie autoryzowało użytkownika, co pozwala na zainicjowanie pobierania danych.
- **Zintegrowane**: Stan docelowy. Odbyło się pobieranie statystyk, a interfejs wyświetla status "Connected".

• **Diagram 2: Cykl życia instancji klasy Dopasowanie (Match)**



- **Zainicjowane:** Stan początkowy nowo utworzonego dopasowania.
- **Aktywne:** Użytkownicy zostali poinformowani, gotowe do rozmowy.
- **Trwający Czat:** Użytkownicy aktywnie korzystają z kanału komunikacji.
- **Zablockowane/Anulowane:** Stany terminalne w przypadku bana od moderatora lub zerwania relacji (unmatch).

• **Diagram 3: Cykl życia dla encji Zgłoszenie**



- **Nowe:** System utworzył obiekt zgłoszenia (inicjalizacja początkowego statusu oraz daty zgłoszenia).
- **Weryfikowane:** Sprawdzenie poprawności danych wprowadzonych w formularzu.

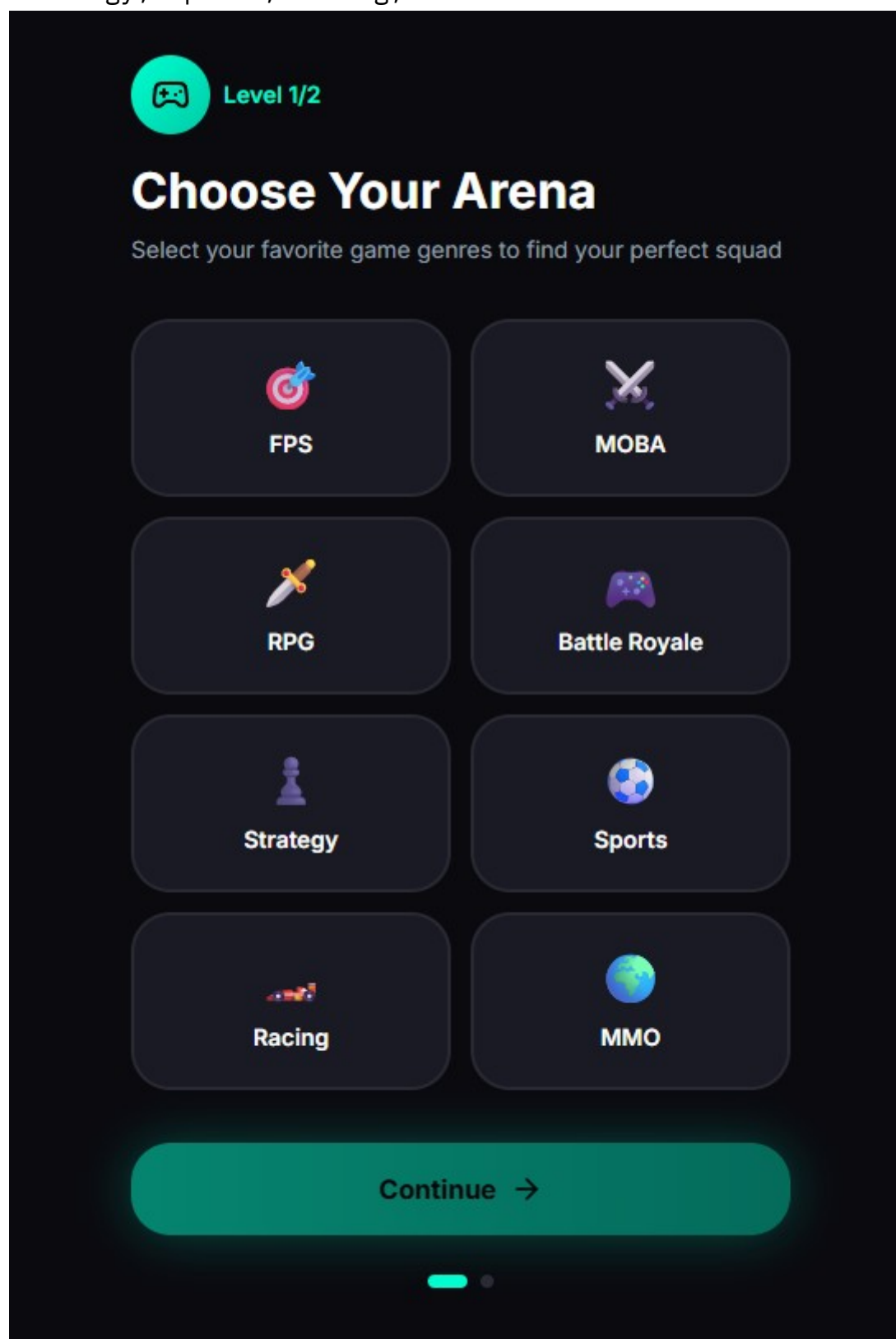
- **Analizowane przez AI:** Analiza treści zgłoszenia przez moduł sztucznej inteligencji, kończąca się zapisem wyniku tej analizy.
- **W kolejce moderatora:** Skierowanie zgłoszenia do listy spraw oczekujących na ręczną weryfikację przez moderatora.
- **Zamknięte:** Stan terminalny następujący po podjęciu decyzji przez Moderatora. System zapisuje końcowy status zgłoszenia (zasadne lub bezpodstawne).

•

5.4. Propozycje interfejsu użytkownika

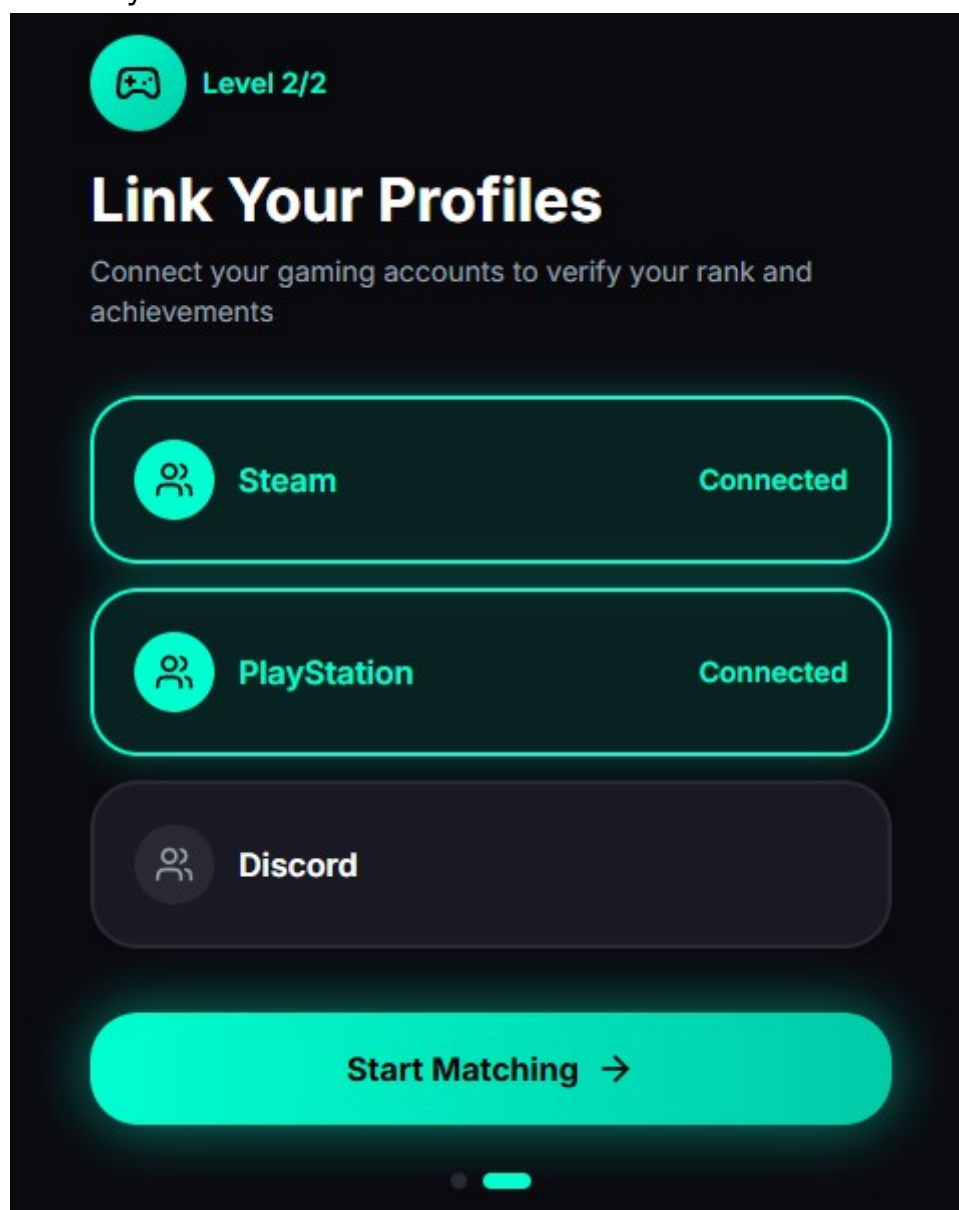
- **Wybór platformy i gatunków gier (Level 1/2):** Ekran "Choose Your Arena", na którym użytkownik proszony jest o wybór preferowanych gatunków gier, aby system mógł lepiej dopasować partnerów. Widoczne opcje: FPS, MOBA, RPG, Battle Royale,

Strategy, Sports, Racing, MMO.



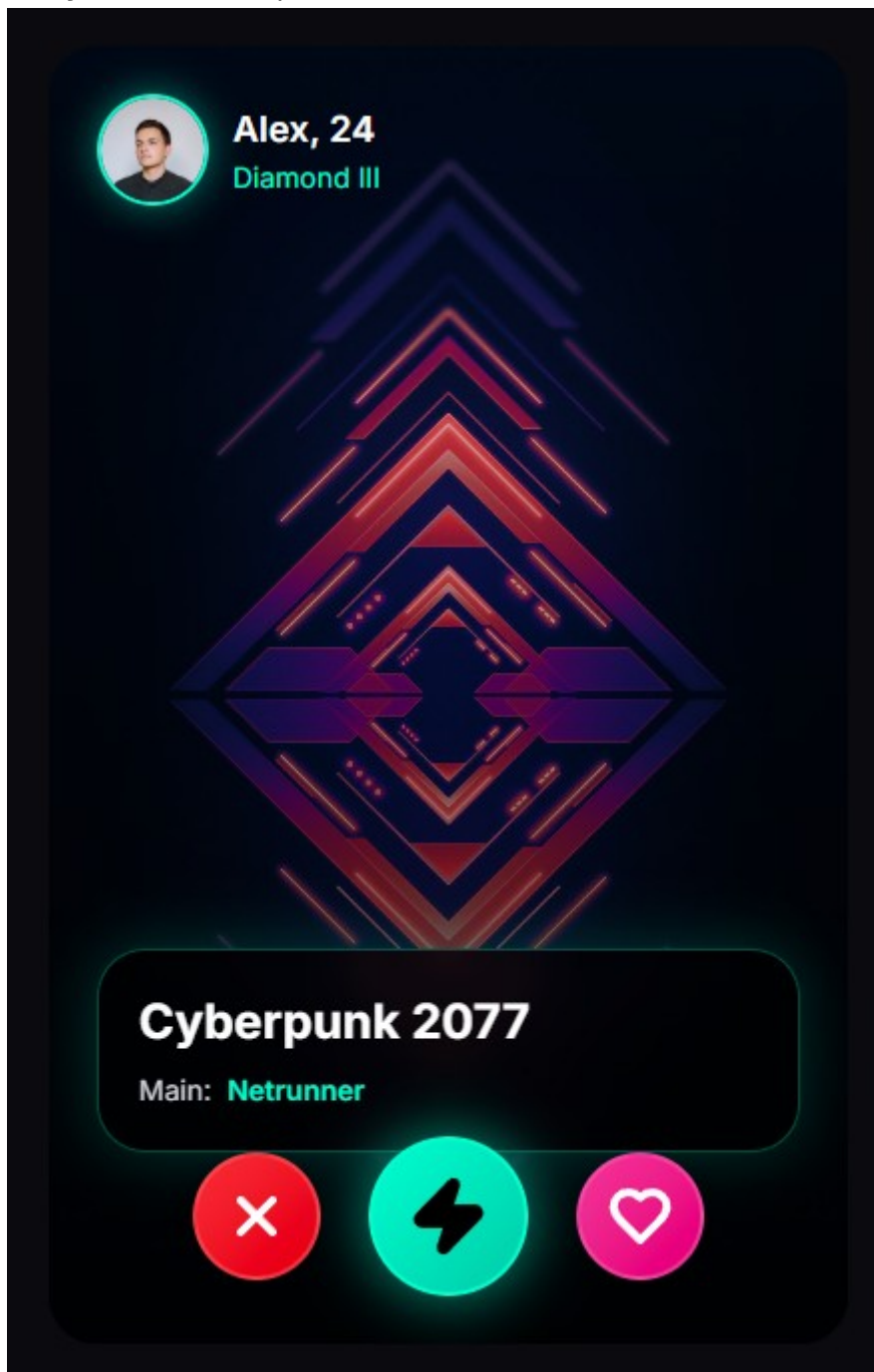
- **Ekran łączenia profili zewnętrznych (Level 2/2):** Ekran "Link Your Profiles" do podłączenia zewnętrznych kont (Steam, PlayStation, Discord) w celu weryfikacji rangi i osiągnięć.

Statusy "Connected".



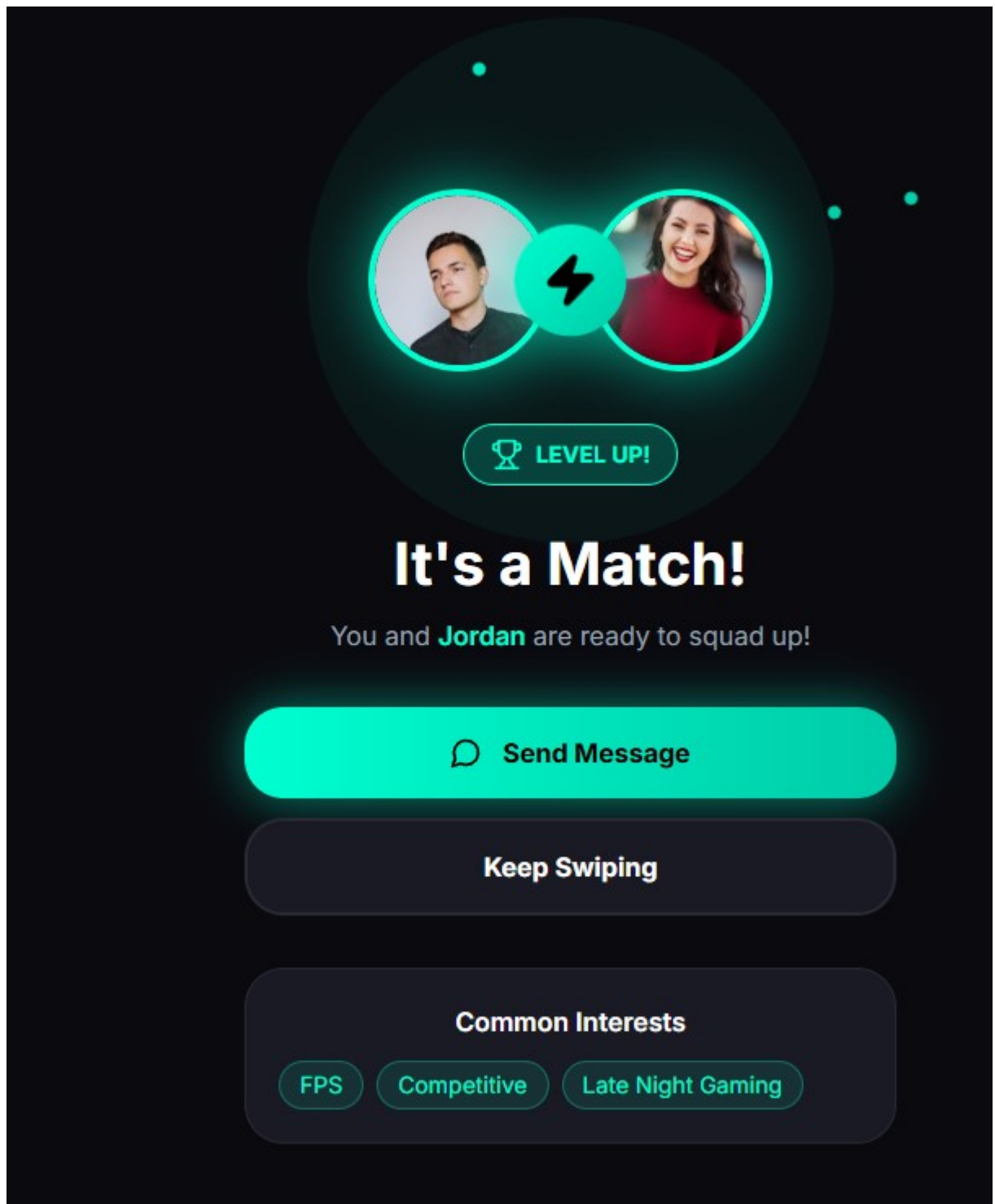
- **Ekran przeglądania (Swipe):** Widoczny profil gracza (nazwa, wiek, ranga), informacja o preferowanej grze i roli. Na dole znajdują się przyciski akcji: odrzucenie (X), supermatch

(błyskawica), polubienie (serce).



- **Ekran Dopasowania (Match):** Informuje o wzajemnym dopasowaniu ("It's a Match!"). Umożliwia natychmiastowe rozpoczęcie rozmowy ("Send Message"), powrót do przeglądania ("Keep Swiping") oraz prezentuje wspólne zainteresowania (Common

Interests).



6. Wymagania нефункционалне dla system

6.1. Oszacowanie wielkości bazy danych

Dla platformy zakładającej 100 000 aktywnych użytkowników, dane profilowe, statystyki pobrane z API zewn. oraz metadane czatu oszacowano na potrzebę początkowej pojemności ~500 GB z rocznym tempem przyrostu o około 300 GB. Należy zastosować partycjonowanie

oraz mechanizmy archiwizacji dla nieaktywnych profili i starszych logów czatu.

6.2. Propozycja wymaganych czasów odpowiedzi

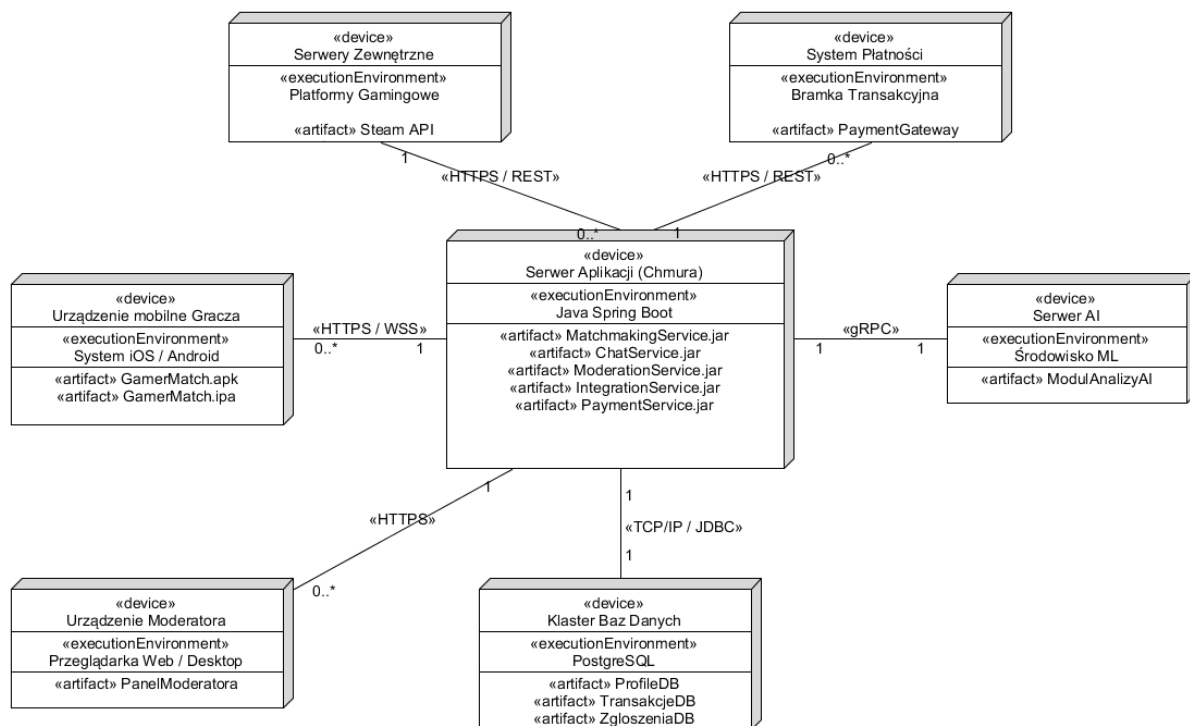
- Analiza zgłoszenia i pobranie wyników w module przeglądania: czas rzędu kilku-kilkunastu sekund na jeden profil.
- Czas połączenia z zewnętrznym API do pobrania danych ranga/statystyki: 30-60 sekund w zależności od serwisu docelowego.
- Zapisanie zgłoszenia użytkownika: typowo ~30 sekund.

6.3. Oszacowanie liczby i typów potrzebnych stanowisk pracy użytkowników systemu

- Stanowiska operatorskie/developerskie: 10 osób dla działów Product & Tech (Programiści Mobile, Backend, DevOps, UI/UX, QA, AI).
- Stanowiska moderacyjne (obsługa 24/7): 3 osoby przypisane wyłącznie do obsługi kolejki zgłoszeń i utrzymywania bezpieczeństwa społeczności.

7. Propozycja technologii informatycznych

- **Sprzęt i środowisko serwerowe (Cloud):** Koszt infrastruktury chmurowej, w tym serwerów i baz danych jest szacowany na 200 000 PLN miesięcznie. Chmura dostarcza usługi maszyn wirtualnych do konteneryzacji.
- **Oprogramowanie i Języki:** Integracje API pisane np. w językach skompilowanych do szybkiego działania (Java/Rust). Moduł analizy obrazów i treści dla moderacji będzie bazował na wytrenowanych modelach AI wykrywających toksyczne zachowania.
- **Diagram wdrożenia:**



8. Propozycja planu pracy Plan pracy przygotowania wersji MVP, sumarycznie trwający 5-6 miesięcy:

- **Etap 1:** Analiza wymagań i projekt systemu (2 tygodnie).
- **Etap 2:** Projekt interfejsu użytkownika UX/UI (3 tygodnie). Wymaga zakończenia Etapu 1.
- **Etap 3:** Implementacja rejestracji, logowania i profilu gracza (4 tygodnie). Alokacja: Zespół Backend + Mobile.
- **Etap 4:** Integracja z platformami Steam/Discord (3 tygodnie). Wymaga istnienia struktury z Etapu 3.
- **Etap 5:** Implementacja mechanizmu dopasowań (5 tygodni). Główny silnik, alokacja: Backend, AI.
- **Etap 6:** Implementacja czatu i powiadomień (4 tygodnie). Zależność po ukończeniu Etapu 5.
- **Etap 7:** Funkcje premium, boost i płatności (3 tygodnie).
- **Etap 8:** Moduł zgłoszeń, moderacji i analizy AI (4 tygodnie).
- **Etap 9:** Testowanie, poprawki i optymalizacja (4 tygodnie). Angażuje QA po wdrożeniach funkcji z etapów 3-8.
- **Etap 10:** Wdrożenie wersji MVP (2 tygodnie). Ostatni etap produkcyjny przed startem.

9. Analiza ryzyka projektu

• **Zagrożenie 1: Niestabilność Zewnętrznych API (Steam/Discord)**

- *Szansa*: Duże. Zewnętrzne systemy regularnie zmieniają reguły ograniczania żądań (rate limiting).
- *Stopień szkodliwości*: Średni (paraliżuje proces integracji nowych kont).
- *Zapobieganie*: Implementacja asynchronicznych kolejek oraz tolerancji na błędy; zapamiętywanie stanów częściowych w cache.
- *Plan awaryjny*: Umożliwienie graczom manualnej weryfikacji przez zrzuty ekranu kontrolowane przez Moderadora.

• **Zagrożenie 2: Zalanie platformy fałszywymi kontami/botami**

- *Szansa*: Średnie.
- *Stopień szkodliwości*: Duży (zaburza dopasowania wewnątrz platformy).
- *Zapobieganie*: Zaawansowane narzędzia analizy zachowań AI (heurystyki) oraz twarde uwierzytelnianie (CAPTCHA).
- *Plan awaryjny*: Wstrzymanie ruchu rejestracyjnego, wdrożenie banowania po adresach IP.

10. Kosztorys realizacji przedsięwzięcia

Łączny czas przygotowania wersji MVP systemu wynosi około 5-6 miesięcy. Przy utrzymaniu miesięcznym zespołu oraz infrastruktury, budżet wynosi 540 000 PLN na miesiąc. Dalszy rozwój po wydaniu MVP może potrwać kolejne 3-6 miesięcy.

Rozbicie na moduły i zespoły (kwoty miesięczne):

- **Dział Product & Tech (10 osób)**: Programiści (Mobile, Backend, DevOps), UI/UX, QA, AI - **300 000 PLN** (obejmuje wynagrodzenia, licencje, sprzęt oraz koszty stanowisk pracy).
- **Dział Marketing & Growth (5 osób)**: Kampanie, influencerzy, pozyskiwanie użytkowników - **120 000 PLN** (obejmuje koszty zespołu oraz miesięczny budżet na kampanie reklamowe).
- **Dział Administracja i Zarząd (2 osoby)**: HR, finanse, prawo - **60 000 PLN**.

- **Dział Moderacja i Wsparcie (3 osoby):** Obsługa zgłoszeń 24/7 - 40 000 PLN.
- **Infrastruktura (Cloud):** Serwery, bazy danych - 20 000 PLN.

Warunki płatności i odbiór:

- Fakturowanie zrealizowanych prac odbywa się w cyklu miesięcznym na podstawie udokumentowanych roboczogodzin oraz zatwierdzonego dostępu do testowych wersji funkcjonalnych na środowisku Staging.
- Ostateczny odbiór wersji MVP potwierdzony pisemnym protokołem odbioru końcowego przez inwestora po okresie ustabilizowanych testów UAT z załogą testową QA. Poziom SLA ustalony na poziomie 99.5%.